Final Project Report

FPGA Musician: Real-Time Audio Effects on the AUP-ZU3

Owen Richmond | EECE 4632 Spring 2026 | Prof. Miriam Leeser | April 23, 2026

1. The Project

A three-effect audio chain (distortion -> tremolo -> delay) running as a custom HLS IP on the FPGA fabric of an AUP-ZU3 Zynq SoC. Audio is 16-bit Q1.15 at 48 kHz, processed in 10 ms chunks of 480 samples. The ARM side runs Python and drives the IP over AXI; the FPGA does all the per-sample math. The actual point of the project was to build the same chain two ways with Vitis HLS and see what the design-space tradeoffs look like: Build 1 is three pipelined loops in one function, Build 2 wraps them in sub-functions with a DATAFLOW pragma. Both are bit-exact against a Python golden reference, both run on the board, both have measured wall-clock numbers. The preliminary report (April 14) covered architecture and a working distortion-only IP; this report picks up from there.

2. Hardware vs. Software

Classic codesign split: the PL handles per-sample math that has to meet a sample-rate deadline; the PS handles control, validation, and anything that doesn't.

Layer	Job	Why there
PL (FPGA)	Three effects in Q1.15. II=1 inner loops. AXI master DMA into DDR.	Tight per-sample work. Deterministic. One sample per cycle once pipelined.
PS (ARM)	Allocate non-cacheable DMA buffers, write parameter regs once, fire AP_START, poll AP_DONE.	No real-time deadline tighter than 10 ms/chunk. Easy to change.
PS (Python tooling)	Golden reference, test harness, WAV I/O, latency measurement.	Off the critical path. Ease of change matters more than speed.

The boundary moved exactly once. I originally planned to do peak normalization in software after each chunk to prevent clipping, but no plausible hardware version of normalization can stay bit-exact with that, so it got dropped and the FPGA uses a hard clip. The Python reference was rewritten to match. Inputs are synthetic: a 440 Hz sine generated at runtime plus an optional WAV file for ear-checking. There is no external dataset.

3. Two Builds of the Same Chain

This is the core of the project. Same three effects, same Q1.15 math, same AXI interfaces. The only structural differences between the two source files are (1) Build 2 lifts each effect into a static sub-function and (2) the top function gets a #pragma HLS DATAFLOW. The loop bodies are character-for-character identical.

Honest scope note: this is two configurations done end-to-end, not a wide sweep. As a one-person semester project I picked the comparison that gets the most signal, sequential pipeline vs overlapping DATAFLOW, and built both through HLS, Vivado, and on-board measurement. Within that scope the

results are clean and the system runs without errors, with enormous real-time headroom. A wider sweep is the next iteration.

Optimizations applied to both builds

Three pragma tricks did most of the work:

1. PIPELINE II=1 on every inner loop. New sample accepted every cycle. The delay loop reads from and writes to the same circular buffer, which a naive scheduler would serialize into II=2 to avoid a RAW hazard. #pragma HLS DEPENDENCE inter false tells HLS the addresses don't alias (the read pointer is wp - delay_n, the write pointer is wp), so it stays at II=1.

2. Hardcoded 64-entry Q1.15 sine LUT for the tremolo LFO. A real sin() in HLS spends a DSP48 on a CORDIC or drags in a shared float unit; a 128-byte ROM costs basically nothing and HLS maps it into distributed LUT memory rather than burning a BRAM tile. Claude generated the values because I was not going to type 64 sines by hand.

3. #pragma HLS RESOURCE on the delay buffer to force a true dual-port BRAM (RAM_2P_BRAM). Without that, HLS can pick a single-port layout that makes II=1 impossible. The delay buffer alone eats 14 of 19 BRAM blocks, which is what would constrain multi-channel scaling.

Build 1 vs Build 2

Build 2 wraps each effect in a sub-function so DATAFLOW has something to schedule across. The intermediate arrays mid1 and mid2 (480 samples each) get auto-promoted from plain scratch BRAMs to ping-pong pairs, so while tremolo is reading chunk N, distortion is already writing chunk N+1 into the other half of the pair. There was a moment of panic about the static LFO phase and delay buffer (DATAFLOW does not love static locals), but synthesis succeeded and hardware is bit-exact per chunk. I wouldn't guarantee state behavior under every scheduler decision; passing state in/out explicitly would be cleaner.

Metric	Build 1	Build 2	Delta
Latency / chunk	1,469 cyc (14.69 us)	1,469 cyc (14.69 us)	unchanged
Initiation Interval	1,470 cyc	493 cyc	3.0x better
LUT	3,249 (4.3%)	3,911 (5.2%)	+20%
FF	2,246 (1.5%)	3,134 (2.1%)	+40%
BRAM	19 (4.0%)	19 (4.0%)	unchanged
DSP	12 (3.3%)	12 (3.3%)	unchanged

DATAFLOW buys 3x throughput in synthesis for ~20% more LUT and ~40% more FF. BRAM and DSP unchanged. The interesting question is whether that 3x actually shows up at the wall clock. Spoiler: mostly no, see section 6.

4. AXI Wiring and the Bring-Up Saga

The IP exposes two AXI interfaces. AXI-Lite (s_axilite) carries control: AP_CTRL plus six 32-bit parameter registers, one transaction at a time. The sample data path is a full AXI master (m_axi) with offset=slave,

meaning the IP itself issues burst reads and writes to DDR at addresses the ARM has handed it. That is what makes it a real DMA instead of programmed I/O.

On the PS side, the Zynq UltraScale+ has HP slave AXI ports that reach the DDR controller directly, bypassing the ARM cache. The IP connects through HP0_FPD via an AXI SmartConnect. Cache-bypass means no flush/invalidate syscall per chunk, which turned out to matter a lot.

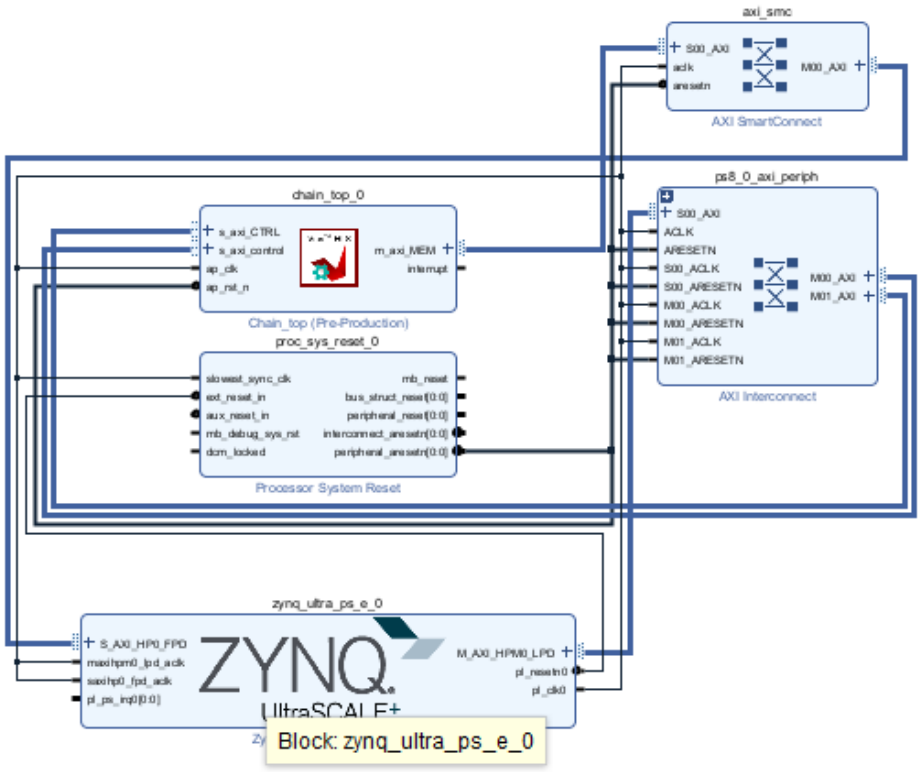

Figure 1. Vivado block design. chain_top_0 has AXI-Lite CTRL on the left and m_axi_MEM on the right going through the AXI SmartConnect into the PS's S_AXI_HP0_FPD. The AXI Interconnect on the right handles the AXI-Lite control path.

Now the part where I lost a weekend. When I imported chain_top into the block design, the AXI-Lite CTRL bus auto-wired through block automation as expected, but the m_axi_MEM port just sat there unconnected. Block automation refused to route it. I clicked every button I could find. On the board this showed up as a completely silent timeout: AP_START would fire, the IP would try a read burst, get nothing back, and never assert AP_DONE. No error. No log message. Just a Jupyter cell that never returned.

Worse, the failure looked exactly like a bug I had hit on the distortion IP weeks earlier (PYNQ wrapping only one of two AXI-Lite buses, also silent), so I burned hours assuming it was the same thing. It was not. The fix was manually enabling S_AXI_HP0_FPD in the Zynq PS customization (PS-PL Configuration > HP Slave AXI Interface), dropping in a SmartConnect, and wiring m_axi_MEM through to HP0 by hand. Why Vivado wouldn't do this on its own is probably an IP-XACT metadata quirk with the HLS export. I stopped

caring once it worked. Total damage: about two days, most of which was assuming the problem was somewhere else.

5. The Python Driver (or: how I made things 13x faster by deleting code)

Once the bitstreams ran, I plugged in the standard PYNQ pattern: cacheable buffers via pynq.allocate, register_map for parameter writes, flush and invalidate around each DMA, poll AP_DONE. The system worked. It was correct. It was also doing 425 us per chunk against 14.69 us of actual FPGA compute time. The FPGA was idle ~96% of the time waiting for Python to set up the next call. Cool.

The optimized driver (pynq_overlay/run_test.py) does three things, each of which is essentially deleting code:

- Allocate buffers with cacheable=False. The HP port sees DDR directly, so no flush/invalidate syscall per chunk.
- Drop register_map for the start path. Direct MMIO into AP_CTRL is one uint32 store instead of a chain of Python attribute lookups.
- Stop writing parameter registers per chunk. They never change during a run; write them once at startup.

Driver	Median latency / chunk	Speedup
Baseline (cacheable, register_map, per-chunk params)	425 us	1.0x
Optimized (non-cacheable, MMIO, static params)	33 us	12.9x

A 13x improvement from removing things. I kept waiting for the catch and there wasn't one. The whole "optimization" took an evening once I understood where the time was going.

6. Testing and Results

Validation runs at three levels: HLS C simulation against a Python-generated reference vector, HLS C/RTL co-simulation against the same vector (this is what confirms the synthesis-report latency and II numbers match real scheduled hardware), and on-board sample-for-sample comparison against an integer-exact Python reference (audio_effects.py "_hls" variants). All three pass on both builds, plus a WAV ear-check that sounds correct end to end.

The on-board test (pynq_overlay/run_test.py) is one script that runs both builds back to back: loads the bitstream, sets parameters once, fires 480-sample chunks at the IP, verifies bit-exactness for one chunk and ten chunks consecutively, measures median and minimum latency over 300 chunks, attempts DATAFLOW-style pipelining, and writes a WAV. The ten-chunk continuity test flags a one-sample divergence at the start of chunk 1 on both builds, almost definately a phase-reset artifact from how the test reloads the Python module rather than a hardware bug. The WAV sounds fine. Worth a cleaner test someday.

Metric	Build 1	Build 2	Real-time req.
Bit-exact (1 chunk, 480 samples)	PASS	PASS	-
Median latency / chunk	42.3 us	33.0 us	< 10,000 us
Min latency / chunk	33.2 us	32.3 us	-

% of 10 ms real-time budget	0.42%	0.33%	< 100%
Sustained throughput (chunks/sec)	23,632	30,331	>= 100
Real-time multiplier	236x	303x	>= 1x

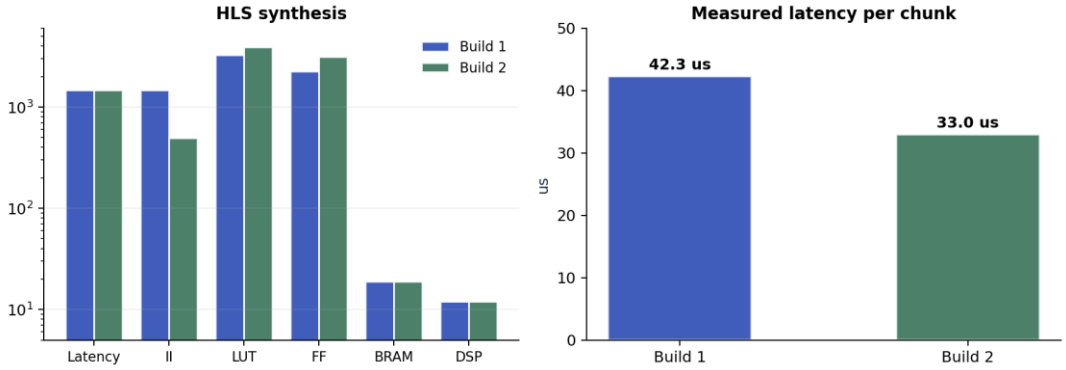

Figure 2. Left: HLS synthesis metrics. Build 2's II drops from 1,470 to 493 cycles (3x throughput) at the cost of more LUT and FF. Right: measured wall-clock latency per 480-sample chunk on hardware, median of 300 runs.

Build 2's measured latency (33 us) is 22% better than Build 1 (42 us), not 3x. The II=493 benefit only shows up if the driver can fire AP_START again within 4.93 us of the previous AP_READY, and Python cannot. The test script measured 0/99 successful pipelined restarts.

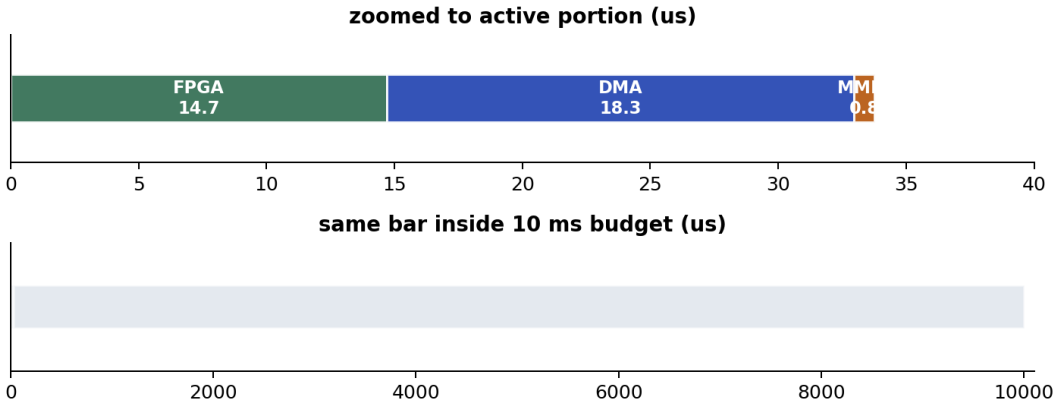

Figure 3. Build 2's 33.8 us active time against the 10,000 us real-time budget. Top: FPGA compute (14.7 us), DMA burst (18.3 us), MMIO/poll (0.8 us), zoomed. Bottom: same bar inside the full 10 ms chunk. The design uses 0.34% of the budget.

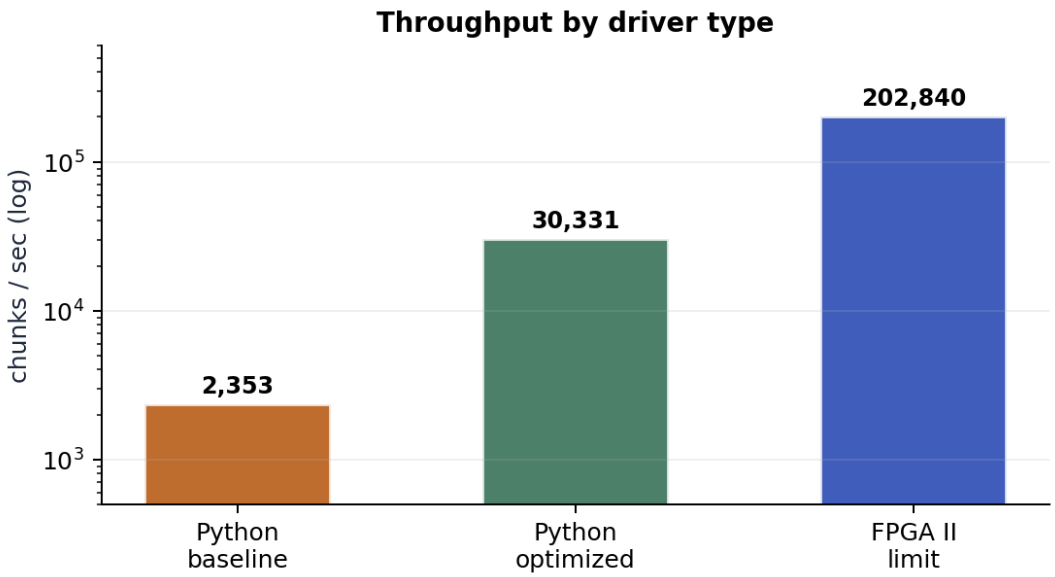

Figure 4. Chunks per second under three drivers: baseline Python, optimized Python, and the FPGA's II limit. Optimized Python is 6x faster than baseline, but a C driver would be another 7x on top of that before the FPGA itself became the bottleneck.

The DATAFLOW gap is the central result. The PL delivered the 3x throughput win in silicon. The PS couldn't exploit it from Python, so most of that win does not show up in wall-clock numbers. For a real-time audio workload at 100 chunks/sec (48 kHz floor), neither build is anywhere close to budget-limited, so it doesn't matter here. If throughput were the goal, the right next move would be a C or baremetal driver, not more PL work.

7. Lessons Learned

Biggest one: the hard parts of a codesign project were not the HLS or the FPGA math. They were AXI plumbing (the m_axi auto-routing thing, plus the earlier PYNQ AXI-Lite issue on distortion) and Python driver overhead. The PL did exactly what it was supposed to do. Almost every debugging hour was spent somewhere else. There is a lesson in there about which parts of a project are actually risky vs which parts feel risky upfront.

Smaller takeaways. Fixed-point is unforgiving; I had to rewrite the Python reference as integer-only to get bit-exactness, because the original did peak normalization the FPGA cannot do. DATAFLOW with static state works in practice but the canonical-form rules are less obvious than the docs make them sound. And bottlenecks move: I started the semester worried about HLS throughput and ended it with the FPGA idle waiting for Python.

What I would do differently.

Build the latency measurement harness on day one, not after the bitstreams work. The Python reference paid off enormously and I built it early; the measurement loop I built late, which is why the 425 us baseline driver caught me by surprise. Also: pass tremolo/delay state in/out as explicit function arguments from the start so DATAFLOW canonical form is guaranteed instead of just empirically correct.

8. If I Had More Time

The semester is over and the project as scoped is done. Both builds work, both run, and the comparison has clear results. In priority order, the next things would be:

1. C/baremetal driver. Closes the 7x throughput gap in figure 4 and would let Build 2's II actually show up at the wall clock. Cleanest single-change improvement available.
2. Stereo. Doubles the BRAM cost (delay buffer is already 14/19 BRAMs, two channels needs 28) but real audio is stereo, so this is the obvious next feature.
3. Refactor for proper DATAFLOW canonical form. Pass LFO phase and delay buffer state explicitly so the scheduler can't reorder against static locals. Closes the "works but I'm not 100% comfortable" caveat from section 3.
4. More effects. The skeleton (Q1.15 chunked, AXI master + AXI-Lite, sub-function DATAFLOW) generalizes to any sample-by-sample effect: biquad EQ, reverb, chorus would all slot in.
5. Live audio I/O. Right now the PS feeds the IP a pre-generated buffer; a real pedal would source from a codec. This is systems integration, not HLS, but it is what would turn the demo into a thing you could plug a guitar into.

9. How I Used AI

I used Claude throughout the project and want to be honest about the scope. Mine, end to end: the architecture (AXI master + AXI-Lite, chunked Q1.15, the two-build comparison), the distortion and chain C++ code, the Python reference integer math, the testbench cases, the fix for the m_axi wiring issue, and the interpretation of the DATAFLOW static-state problem. The actual debugging was all manual.

Claude-assisted, under direction: the Build 2 sub-function refactor (I specified the architecture, Claude did the mechanical rewrite, credited in-file), the 64-entry Q1.15 sine LUT values (I specified the format, Claude generated the numbers, I spot-checked), most of the hardcoded default parameters in run_test.py, the print formatting in the test output, all of the matplotlib graph code in make_graphs.py, and the python-docx plumbing for this report. The numbers behind the graphs and tables come from real measurements; the plotting code does not.

10. Code Submission

Full project on GitHub. Per the assignment guidance, code is submitted alongside this report rather than embedded. Layout and run instructions are in README.md at the project root: vitis_hls/build1_pipeline/ and vitis_hls/build2_dataflow/ for the HLS sources and reports, pynq_overlay/ for the bitstreams and driver, python/ for the standalone reference, FINAL REPORT/ for figures and the source of this document. Inputs are generated at runtime so there is no external dataset to download.

11. References

[1] AMD/Xilinx. Vitis High-Level Synthesis User Guide UG1399, 2024.1. https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls (accessed April 2026).

[2] AMD/Xilinx. pragma HLS dataflow - UG1399. https://docs.amd.com/r/en-US/ug1399-vitis-hls/pragma-HLS-dataflow (accessed April 2026).

[3] AMD/Xilinx. AXI Reference Guide UG1037. https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide (accessed April 2026).

[4] AMD/Xilinx. Zynq UltraScale+ Device TRM UG1085: HP Slave AXI Interfaces. https://docs.amd.com/r/en-US/ug1085-zynq-ultrascale-trm (accessed April 2026).

[5] AMD University Program. AUP-ZU3 Getting Started Guide. https://xilinx.github.io/AUP-ZU3/getting_started.html (accessed April 2026).

[6] PYNQ Project. Supported Boards and Pre-Built Images. http://www.pynq.io/boards.html (I ran this on my machine at home, sorry 😟